Learning Statistically Accurate Dynamics of Complex Systems

UW–Madison Mathematics Experiential Learning (MXM) | Spring 2026

Research Question and Motivation

Existing latent data-assimilation methods combine an autoencoder, an LSTM, and an ensemble Kalman filter to correct high-dimensional forecasts using sparse pointwise physical pseudo-observations. However, scientifically useful information is often available only in aggregate form, such as means, variances, correlations, energy statistics, or event frequencies. Unlike pointwise observations, these statistics do not uniquely identify the underlying physical state: many different state configurations can produce the same collection of moments. Directly filtering the full probability distribution is also computationally difficult because the unknown distribution is an infinite-dimensional object.

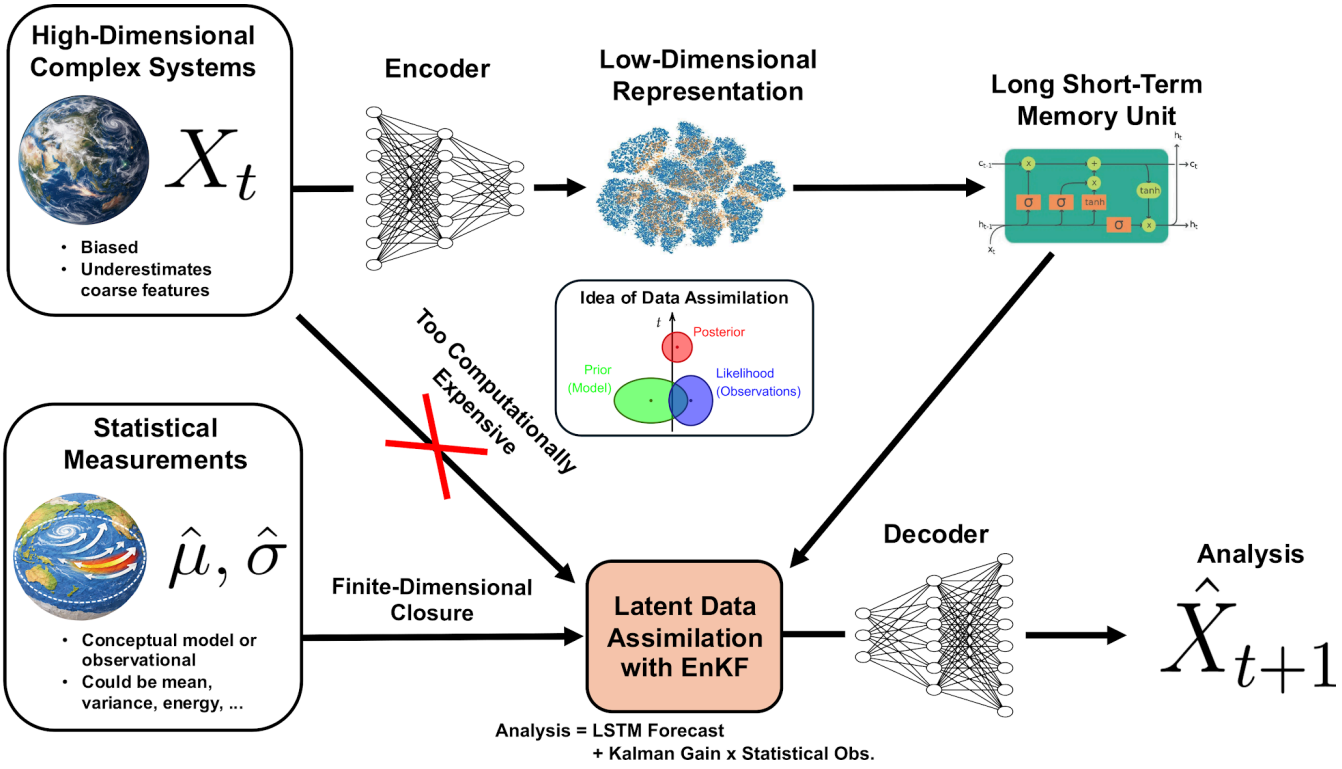
This project investigates a tractable latent-space extension of the existing framework. An autoencoder compresses the physical state, while its decoder provides a learned closure connecting latent variables to plausible full-state reconstructions. An LSTM supplies a short-term dynamical forecast that restricts the correction to temporally plausible states. Statistical observations are coupled with the latent representation so that a proposed EnKF update can use discrepancies between predicted and observed statistics to correct the latent forecast.

Research question: Can statistical observations be assimilated into a learned low-dimensional dynamical model so that they meaningfully constrain the reconstructed high-dimensional system, preserve plausible short-term dynamics, and guide the evolving ensemble toward the correct long-term statistical behavior?

The objective is therefore not necessarily to recover one exact trajectory indefinitely. For chaotic stochastic systems, the more appropriate goal is to retain useful short-term state information while reproducing the correct invariant distribution or statistical attractor.

Proposed Full Framework

The project-wide design encodes a high-dimensional physical state, forecasts its low-dimensional representation with an LSTM, combines that forecast with physical or statistical observations through a proposed latent-space EnKF update, and decodes the corrected representation back to the physical state. The diagram below shows the intended cycle.



Important status note: this diagram presents the intended EnKF-based framework. It is not evidence that the complete EnKF stage was implemented in the reported experiments.

Choice of Case-Study Models

The project applies the framework to the stochastic Lorenz '96 (L96) model and the stochastically coupled FitzHugh–Nagumo (FHN) model, two high-dimensional chaotic systems commonly used as testbeds for data assimilation, prediction, and uncertainty quantification methods in the context of complex systems. Lorenz '96 is a classical conceptual model that mimics the propagation of atmospheric waves on a midlatitude zonal circle and exhibits chaotic and turbulent features. FitzHugh–Nagumo is a classical simplified neuronal dynamical system that captures the dynamics of action potentials. Its time-scale separation creates the fast–slow dynamics characteristic of neuronal spiking, while the spatially coupled model can exhibit traveling waves and extended spatial patterns through the propagation of action potentials. The two case studies therefore examine the framework across atmosphere-like chaotic dynamics and neuronal fast–slow excitable dynamics.

Lorenz–96 Model

The Lorenz–96 case study uses a stochastic system with 40 state variables and periodic indexing:

$$dx_i = [x_{i-1}(x_{i+1} - x_{i-2}) - x_i + F]dt + \sigma dW_i(t), \quad i = 1, \ldots, 40.$$

- The nonlinear term transports energy around the periodic lattice.
- The $-x_i$ term represents damping.
- F represents external forcing.
- The stochastic term adds random forcing or noise.
- The model loosely represents atmospheric energy dynamics.

- The trajectory was simulated using the Euler–Maruyama numerical method for stochastic differential equations.

Overleaf evidence: main.tex, "Stochastic Lorenz '96 Model"; Lorenz_Autoencoder_Walkthrough.tex, "Data Generation."

Completed Lorenz–96 Workflow —Baseline Model

1. Data generation and preprocessing

We first simulated how the 40 interacting variables changed over time. The simulation used Euler–Maruyama, a standard numerical method for systems affected by random noise. Before training the neural network, we scaled the variables to comparable numerical ranges so variables with naturally larger values would not dominate the model's learning. We used one fixed adjustment throughout the time sequence, allowing the model to retain meaningful changes in the system over time.

2. Augmented convolutional autoencoder

The completed Lorenz–96 **baseline** uses a one-dimensional convolutional autoencoder in PyTorch. Each normalized physical state $x_t \in \mathbb{R}^{40}$ passes through the encoder, producing eight learned coordinates $z_t = E(x_t) \in \mathbb{R}^{8}$. Five selected pointwise physical pseudo-observations $o_t \in \mathbb{R}^{5}$ are extracted from the same state and appended to z_t, forming the augmented state $a_t = [z_t ; o_t] \in \mathbb{R}^{13}$. The decoder receives this full augmented state and reconstructs the physical field, $\hat{x_t} = D(a_t) \in \mathbb{R}^{40}$.

- Periodic geometry: circular padding respects the Lorenz–96 lattice.
- Encoder architecture: three stride-2 convolutional blocks reduce the spatial grid from 40 to 20 to 10 to 5, followed by a fully connected projection to eight learned coordinates.
- Pointwise observations: five values are selected at zero-based code indices [1, 5, 10, 15, 35]. They are individual physical values, not statistical moments.
- Augmented representation: eight learned coordinates plus five pointwise observations produce the 13-dimensional decoder input.

3. Autoencoder training objective

The documented objective combines mean squared reconstruction error with a cross-correlation regularizer. The regularizer encourages statistical coupling between learned coordinates and the appended observations so that a later assimilation update could propagate through the latent state. Documented configuration: Adam optimizer, learning rate 5×10^{-4}, batch size 128, and 50 epochs.

4. Latent-dynamics LSTM

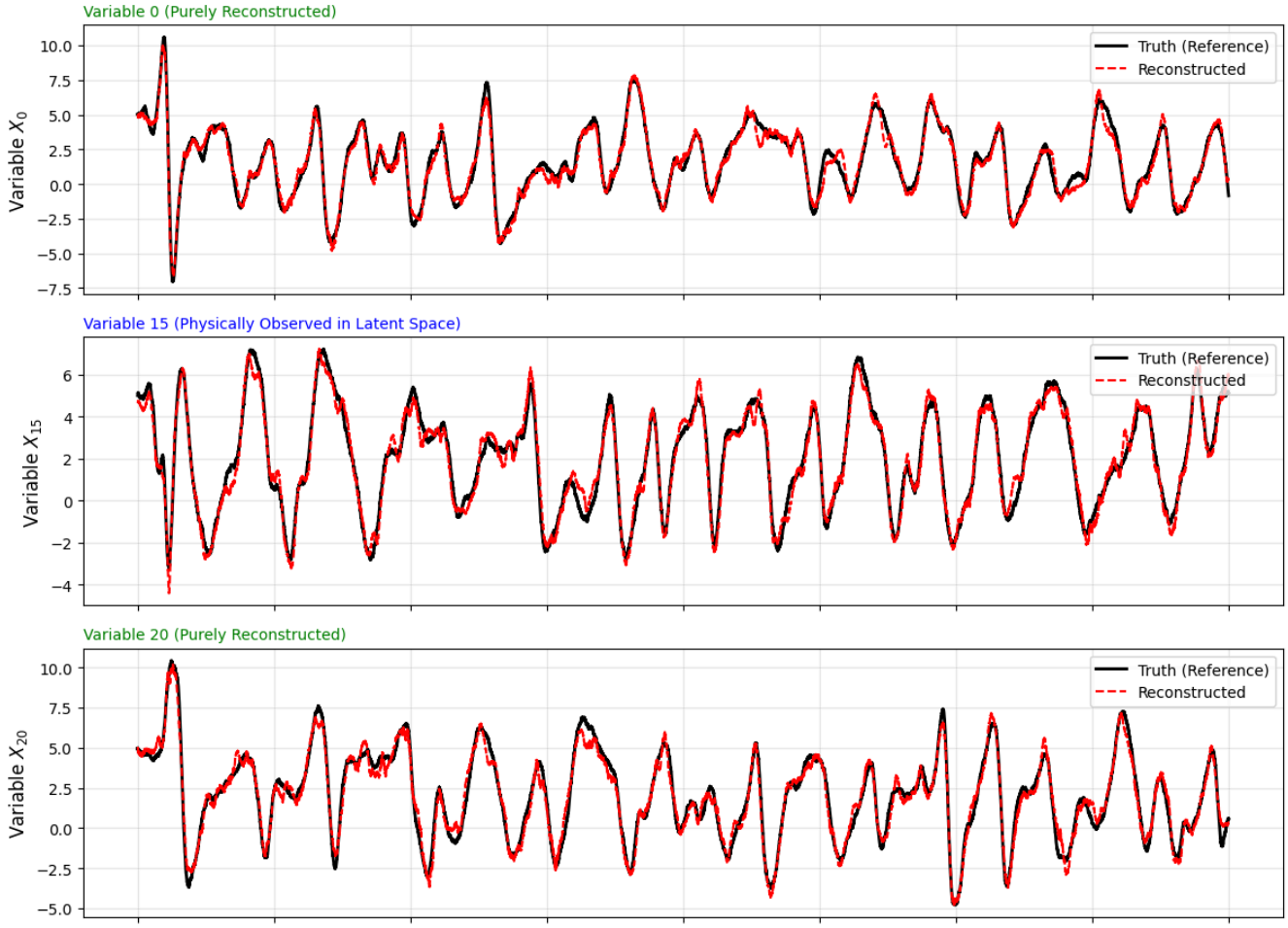
After autoencoder training, the encoder is frozen and applied to the trajectory. The five observations are appended again to create a 13-dimensional time series for LSTM training.

- Input: three previous augmented latent states.
- Output: the next augmented latent state.
- Architecture: two LSTM layers with hidden dimension 32.
- Training: normalized latent-space mean squared error, Adam optimizer, learning rate 10^{-4}, and 50 epochs.
- Evaluation: one-step encode–forecast–decode testing and an autoregressive rollout benchmark.

Reported Results

Autoencoder reconstruction

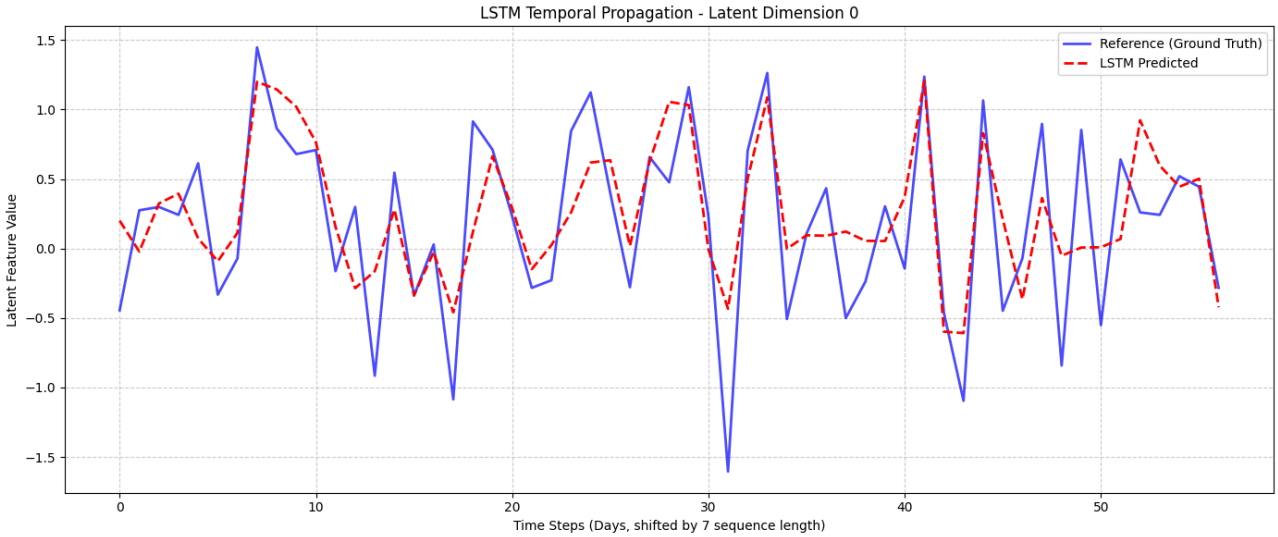
The final report compares reconstructed Lorenz–96 variables with reference trajectories. Variable 15 is one of the appended physical observations; variables 0 and 20 are reconstructed without being directly appended. The curves track closely in the displayed examples.

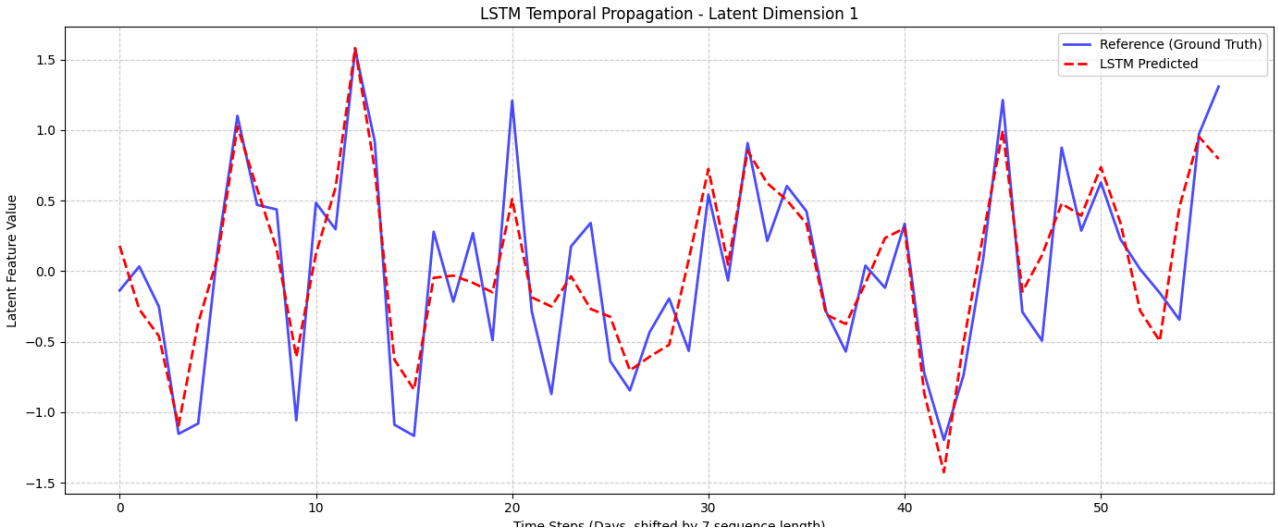


Evidence-supported interpretation: the autoencoder qualitatively reconstructed both observed and unobserved Lorenz–96 components. No numerical reconstruction error or test-set improvement is reported.

Latent LSTM forecast

The LSTM figure compares predictions with encoded reference trajectories for latent dimensions 0 and 1. The predictions reproduce much of the general temporal behavior, with visible deviations, particularly in latent dimension 0.





Evidence-supported interpretation: the LSTM qualitatively reproduced short-term latent dynamics. The materials provide no RMSE, accuracy percentage, or baseline comparison.

Statistical-Observation Extension—Improved model

1. Constructing statistical observations from the physical state

Our project investigates how to incorporate statistical information into a learned representation of a dynamical system. Instead of describing the system through a few individual state values, statistical observations summarize properties across multiple locations.

For Lorenz–96, each snapshot contains 40 physical variables. We define a statistical observation operator, M, that calculates selected summaries from that snapshot:

$m_t = M(x_t)$.

Here, x_t is the physical state at time t, and m_t is the resulting collection of statistical observations. Applying the same calculation to successive snapshots produces a time series of statistics.

One documented operator calculates five spatial block averages. It divides the 40 variables into five consecutive groups of eight and averages each group. Using the code's zero-based indexing, the first average is

$$m_{t,1} = [x_0(t) + x_1(t) + \cdots + x_7(t)] / 8.$$

The remaining averages summarize variables 8–15, 16–23, 24–31, and 32–39. Each observation therefore represents the average state of a small region of the system. These are averages across locations at one time; repeating the calculation allows us to track how those regional averages evolve.

Another documented operator extracts the real and imaginary components of the first three nonzero spatial Fourier modes. These six numbers summarize large-scale patterns around the Lorenz–96 ring. Additional operators calculate quantities such as the spatial mean, variance, and mean squared amplitude.

In these simulated examples, the statistical observations are calculated directly from the simulated physical states. Their meanings are chosen explicitly through M, while the encoder's latent coordinates are learned during training.

2. Approach 1: incorporating statistics through an augmented representation

The first approach explicitly includes the statistical observations alongside the encoder's learned coordinates. In the documented Lorenz block-average configuration, the encoder compresses the 40-dimensional state into eight learned coordinates. The five block averages are calculated separately from the same input and appended to those coordinates:

$$a_t = [E(x_t); M(x_t)].$$

The resulting representation contains eight learned coordinates plus five statistical observations, giving 13 values in total. The decoder receives all 13 values and reconstructs the original 40-dimensional state.

The two parts of this representation serve different purposes. The learned coordinates encode features useful for reconstruction, while the appended statistics explicitly retain selected, interpretable properties of the physical state. Because both are derived from the same training snapshot, appending the statistics makes those properties directly available to the model without introducing an independent measurement.

Training combines physical-state reconstruction error with a correlation regularizer. Reconstruction error measures how closely the decoder's 40 outputs match the 40 input values. The regularizer encourages relationships between the learned coordinates and the statistical observations across a batch of samples. For example, it encourages the learned representation to reflect changes associated with a rising or falling regional average.

This coupling matters because we want the statistical quantities to be connected to the learned representation, rather than simply stored beside it. The regularizer encourages that connection, although its effectiveness still needs experimental evaluation.

The LSTM is then designed to forecast the augmented representation over time. In the block-average configuration, it would predict the next eight learned coordinates and five statistical coordinates from their recent history. The 13-dimensional size is specific to this configuration: using six Fourier observables with eight learned coordinates would instead produce 14 dimensions.

The walkthrough documents this Lorenz extension configuration and its training calculations; the displayed Lorenz result figures do not establish the performance of this statistical version.

3. Approach 2: learning statistics as functions of the latent state

The alternative approach retains only the eight learned coordinates in the reduced state. A separate neural network, MomentNet, maps those coordinates to the selected statistical quantities. In this configuration, the decoder also reconstructs the physical state directly from the eight learned coordinates.

This creates two ways to obtain statistics from a latent state:

- Decode the physical state and calculate its statistics.
- Use MomentNet to predict those statistics directly.

The training design connects these two routes:

$$h(z) \approx M(D(z)),$$

where h is MomentNet, D is the decoder, and M calculates the selected summaries.

The objective combines three errors:

Training term	What it compares	Purpose
Physical reconstruction error	Decoded state with the reference physical state	Preserve the physical information needed for reconstruction.
Statistical reconstruction error	Statistics of the decoded state with reference statistics	Encourage the reconstructed field to preserve the chosen summaries.
Observation-map consistency error	MomentNet's outputs with statistics of the decoded state	Keep the directly predicted statistics consistent with the reconstructed physical field.

For example, suppose the reference regional average is 2.0. MomentNet might predict 2.0 while the decoded physical field has an average of 1.5. Correctly predicting the reference average would not, by itself, mean the reconstructed field had the correct average. The second training term penalizes the decoder's statistical error, while the third penalizes disagreement between MomentNet and the decoder.

The walkthrough provides a MomentNet example with two hidden layers of 64 units and five block-average outputs. This approach keeps the evolving representation smaller, while requiring an additional nonlinear network and training objectives that coordinate its outputs with the decoder.

The report states that alternative approaches were tested, but it does not provide a separate quantitative comparison establishing which approach performed better.

4. Reported application of statistical augmentation

The team's FitzHugh–Nagumo study provides reported results using statistical augmentation. Its physical state contains two fields, u and v, with 40 variables each. The poster describes compressing this 80-dimensional state into eight learned coordinates and training an LSTM with four appended statistical features.

Examples consistently identified in the report and poster include the spatial mean of u, the correlation between u and v, and the fraction of u values exceeding a spike threshold. The spike fraction describes how much of the system is spiking at a particular time. For example, if 10 of the 40 locations exceed the threshold, the fraction is 10/40 = 0.25.

The report presents latent forecasts, decoded physical time series, and the spike fraction calculated from decoded LSTM predictions. Comparing this fraction with the reference shows whether the predicted physical field reproduces the collective spiking activity of the system. The displayed curves capture many spiking periods, with visible differences in some peaks.

These results document statistical-feature forecasting in the team's FHN study. The spike-statistics figure uses training data as its reference, so it provides qualitative evidence rather than a demonstrated improvement on unseen data or a controlled comparison with the pointwise approach.

Ensemble Kalman Filter Status—Future Focus

The Ensemble Kalman Filter (EnKF) is the proposed stage that uses statistical observations to adjust the LSTM's forecasts.

1. Represent uncertainty with an ensemble

The filter maintains a collection of possible reduced states, called an ensemble. Each member represents one possible estimate of the system.

For the five-block-average configuration, the state being updated depends on the approach:

Approach	State carried by each ensemble member
Statistical augmentation	Eight learned coordinates plus five statistical coordinates: **13D**
MomentNet	Eight learned coordinates: **8D**

Each member also has a recent history of reduced states, which provides the input required by the LSTM. Differences between the members represent uncertainty in the estimated state.

2. Use the LSTM to forecast each member

The LSTM advances each member's history to produce its next reduced-state prediction. Together, these predictions form the forecast ensemble.

This is the model's estimate before incorporating the statistical observations available at that time. The walkthrough also allows optional forecast perturbations to represent model uncertainty; these are set to zero in its minimal example.

3. Calculate what statistics each forecast predicts

To compare a reduced-state forecast with statistical observations, the filter needs a corresponding prediction of those same statistics.

In Approach 1, the predicted statistics are already included in the augmented state. The observation operator selects the five appended statistical coordinates.

In Approach 2, the forecast contains only eight learned coordinates. MomentNet maps them to predicted statistical observations.

For example, if the observed quantity is a regional average, the filter compares the observed regional average with the regional average predicted through the relevant observation operator.

The difference is called the innovation:

innovation = observed statistics − predicted statistics.

If a reference block average is 2.0 and a member predicts 1.0, that component of the innovation is +1.0. The filter must then determine how this discrepancy should change the reduced-state estimate.

4. Use ensemble relationships to determine the correction

The EnKF estimates how changes in the reduced coordinates are associated with changes in the predicted statistics across the ensemble. These relationships are summarized by cross-covariance.

For example, ensemble members with a larger value of a particular latent coordinate might also predict a larger regional average. An observed average above the forecast can then support increasing that coordinate. Other coordinates may move differently, depending on their relationships with the observations.

The Kalman gain, K, converts the statistical discrepancies into adjustments to the reduced state:

$K = P_{sm}(P_{mm} + R)^{-1}$.

Here:

- P_{sm} describes how reduced states and predicted statistics vary together.
- P_{mm} describes the ensemble uncertainty in the predicted statistics.
- R describes observation-error uncertainty.

The gain is calculated from the forecast ensemble and observation uncertainty at each update. Its effectiveness depends on the ensemble representing useful relationships between the state and the observations.

The stochastic EnKF documented in the walkthrough updates each member using:

$s_i^a = s_i^f + K(m^{obs} + \eta_i - \hat{m}_i^f)$.

In this expression, s_i^f is the member's forecast, s_i^a is its updated estimate, and $\hat{m}_i^f$ contains its predicted statistics. The perturbation η_i is sampled according to the observation-error covariance R, allowing the update to account for observation uncertainty.

5. Decode the updated state and continue forecasting

The decoder maps the updated reduced state back to the full physical state. For Lorenz–96, this produces an updated estimate of all 40 physical variables.

Each member's updated reduced state is also appended to its LSTM history, replacing the oldest entry. The next forecast therefore starts from the updated history.

This produces the repeating sequence:

Forecast → predict statistics → compare with observations → update the reduced state → reconstruct the physical state.

The correction changes the estimated state; this stage does not retrain the autoencoder or LSTM.

Verified Tools and Methods

- Python and PyTorch.
- NumPy-style numerical simulation and preprocessing.
- Euler–Maruyama stochastic integration.
- One-dimensional convolutional autoencoder with circular padding.
- LSTM latent-dynamics forecasting.
- Mean squared error, cross-correlation regularization, and Adam optimization.
- Qualitative trajectory reconstruction and latent-forecast visualization.

Open Items to Verify Before Public Use

- Forcing value: the final report says $F = 8$, while the poster and walkthrough say $F = 5$.
- Simulation length: the poster says 80,000 time points, while the walkthrough configuration $T = 40$ and $\Delta t = 0.001$ produces 40,000 steps.

Until the original experiment or notebook resolves these discrepancies, neither value should appear as a definitive portfolio claim.